/*

Set Operators: Set operators combine results from two or more queries into a single result set. SQL Server provides the following set operators.

- UNION
- UNION ALL
- INTERSECT
- EXCEPT

To combine the results of two queries we need to follow the below basic rules.

- The number and the order of the columns must be the same in all queries.
- The data types must be compatible(Well-Matched)

Example:

CREATE TABLE EMP_HYD (EID INT, ENAME VARCHAR (50), SALARY MONEY)

CREATE TABLE EMP_CHENNAI (EID INT, ENAME VARCHAR (50))

EMP_HYD

EID	ENAME	SALARY
101	SAI	25000.00
102	SIDDHU	32000.00
103	KAMAL	42000.00
104	NEETHU	63000.00

EMP_CHENNAI

EID ENAME

101 SAI

105 POOJA

106 JASMIN

UNION: it combines the result of two or more selec

t statements into a single result set that includes all the

records belongs to all queries except duplicate values.

Ex: Select Ename from EMP_HYD Union

Select Ename from EMP_CHENNAI

OUTPUT: ENAME

JASMIN KAMAL NEETHU POOJA SAI SIDDHU

UNION ALL: it is same as union but returns duplicate values

Ex:

Select Ename from EMP_HYD

Union ALL

Select Ename from EMP_CHENNAI

OUTPUT: ENAME SAI SIDDHU KAMAL NEETHU SAI POOJA JASMIN

INTERSECT: INTERSECT returns any distinct

values that are common in left and right tables.

Ex: Select Ename from EMP_HYD Intersect

Select Ename from EMP_CHENNAI

OUTPUT: ENAME SAI

EXCEPT: EXCEPT returns any distinct values from the

left query that are not found on the right query.

Ex: Select Ename from EMP_HYD Except

Select Ename from EMP_CHENNAI

OUTPUT: ENAME KAMAL NEETHU SIDDHU

Ex: Select Ename from EMP_CHENNAI Except

Select Ename from EMP_HYD

OUTPUT: ENAME JASMIN POOJA

CLAUSES IN SQL:

We can add these to a query for adding additional options like filtering the records, sorting records and grouping the records with in a table. These clauses contains the following clauses are,

WHERE: This clause is used for filter or

restricts the records from the table.

Ex: SELECT * FROM EMP WHERE SAL=10000

ORDER BY: The order by clause is used to sort or

arrange the data in ascending or descending order with

in table. By default order by clause arrange or sort

the data in ascending order only.

- If we want to arrange the records in a descending order then we use Desc keyword.
- We can apply order by clause on integer and

string columns.

Ex: SELECT * FROM EMP ORDER BY EID (For Ascending Order)

Ex: SELECT * FROM EMP ORDER BY ENAME DESC (For Descending Order)

TOP N CLAUSE: This clause is used to fetch a top n number

of records from a table.

Ex: SELECT TOP 3 * FROM EMP

Ex: UPDATE TOP 3 EMP SET ENAME="SAI"

Ex: DELETE TOP 3 FROM EMP

GROUP BY: Group by clause will use for to arrange similar data into groups. when we apply group by clause in the query then we use group functions like count(),sum(),max(), min(),avg().

If we use group by clause in the query, first the data in the table will be divided into different groups based on the columns and then execute the group function on each group to get the result.

Ex1: WAQ to find out the number of employees working in the organization

Sol: SELECT COUNT (*) FROM EMP

Ex2: WAQ to find out the number of employees working in each group in the organization.

Sol: SELECT DEPT, COUNT=COUNT (*) FROM EMP GROUP BY DEPT

Ex3: WAQ to find out the total salary of each department in the organization

Sol: SELECT DEPT, TOTALSALARY=SUM (SALARY) FROM EMP GROUP BY DEPT (Like this we can find max, min, avg salary in the organization)

HAVING CLAUSE: Having clause is also used for filtering and restricting the records in a table just like where clause.

Ex: WAQ to find out the number of employees in each department only if the count is greater than 3

Sol: SELECT DEPT, COUNT=COUNT (*) FROM EMP GROUP BY DEPT
HAVING COUNT (*) >3

Differences Between WHERE and HAVING Clause:

WHERE HAVING

WHERE clause is used to filter and

restrict the records before grouping HAVING clause is used to filter and

restrict the records after grouping

If restriction column associated with

A aggregative function then we cannot use WHERE clause there But we can use HAVING clause at this situations

WHERE clause can apply without group

by clause HAVING clause cannot be applied

without a group by clause

WHERE clause can be used for restricting individual rows Where as HAVING clause is used along

with group by clause to filter or restrict groups

Constraint in SQL

Why Constraint in SQL: Constraint is using to restrict the insertion of unwanted data in any columns. We can create constraints on single or multiple columns of any table. It maintains the data integrity i.e. accurate data or original data of the table. Data integrity rules fall into three categories:

- Entity integrity
- Referential integrity
- Domain integrity

Entity Integrity: Entity integrity ensures each row in a

table is a uniquely identifiable entity.

You can apply entity integrity to a table by specifying a

PRIMARY KEY and UNIQUE KEY constraint.

Referential Integrity: Referential integrity ensures

the relationships between tables remain preserved as

data is inserted, deleted, and modified. You can apply

referential integrity using a FOREIGN KEY constraint.

Domain Integrity: Domain integrity ensures
the data values inside a database follow defined rules
for values, range, and format. A database can enforce
these rules using CHECK KEY constraints.

Types of constraints in SQL Server:-

1. Unique Key constraint.
2. Not Null constraint.
3. Check constraint
4. Primary key constraint.
5. Foreign Key constraint.

1. Unique Key:- Unique key constraint is use to
make sure that there is no duplicate value in that column.
Both unique key and primary key both enforces the
uniqueness of column but there is one difference between
them unique key constraint allow null value but
primary key does not allow null value.

In a table we create one primary key but we can create more than one unique key in Sql Server.

Ex: create table EMP(EID int unique,ENAME varchar(50) unique,SALARY money);

2. Not null constraint: - Not null constraint is used to restrict the insertion of null value at that column but allow duplicate values.

Ex: create table EMP(EID int not null,ENAME varchar(50) not null,SALARY money);

3. Check Constraint: - This constraint is using to check value at the time of insertion like as salary of any employee is always greater than zero. So we can create a check constraint on employee table which is greater than zero.

Ex: create table emp4(eno int,ename varchar(50),age int check (age between 20 and 30))

4. Primary Key:- Primary key is a combination of unique and not null which does not allow duplicate as well as null values into a column. In a table we create one primary key only.

Ex:create table emp(EID int primary key,ENAME varchar(50),SALARY money)

5. Foreign Key: - One of the most important concepts in database is creating relationships between database tables. These relationships provide a mechanism for linking data stored in multiple tables and retrieving it in an efficient manner.

In order to create a link between two tables we must specify a foreign key in one table that references a column in another table.

Foreign key constraint is used for relating or binding two tables with each other and then verifies the existence of one table data in the other.

To impose a foreign key constraint we require the following things.

We require two tables for binding with each other and those two tables must have a common column for linking the tables.

JOINS IN SQL: Joins are used for retrieving the data from more than one table at a time. Joins can be classified into the following types.

- EQUI JOIN
- INNER JOIN
- OUTER JOIN
- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN
- NON EQUI JOIN
- SELF JOIN

- CROSS JOIN
- NATURAL JOIN

EQUI JOIN: If two or more tables are combined using equality condition then we call as an Equi join.

Ex: WAQ to get the matching records from EMP and DEPT tables

Sol: SELECT * FROM EMP, DEPT WHERE (EMP.EID=DEPT.DNO) (NON- ANSI STANDARD)

Sol: SELECT E.EID, E.ENAME, E.SALARY, D.DNO, D.DNAME FROM EMP E, DEPT D WHERE E.EID=D.DNO (NON-ANSI STANDARD)

INNER JOIN: Inner join return only those records that match in both table

Ex: SELECT * FROM EMP E INNER JOIN DEPT D ON E.EID=D.DNO (ANSI)

OUTTER JOIN: It is an extension for the Equi join. In Equi join condition we will be getting the matching data from the tables only. So we loss UN matching data from the tables.

To overcome the above problem we use outer join which are used to getting matching data as well as UN matching data from the tables. This outer join again classified into three types

LEFT OUTER JOIN: It will retrieve or get matching data from both table as well as UN matching data from left hand side table

Ex: SELECT * FROM EMP LEFT OUTER JOIN DEPT ON EMP.EID=DEPT.DNO;

RIGHT OUTER JOIN: It will retrieve or get matching data from both table as well as UN matching data from right hand side table

Ex: SELECT * FROM EMP RIGHT OUTER JOIN DEPT ON EMP.EID=DEPT.DNO;

FULL OUTER JOIN: It will retrieve or get matching data from both table as well as UN matching data from left hand side table plus right hand side table also.

Ex: SELECT * FROM EMP FULL OUTER JOIN DEPT ON EMP.EID=DEPT.DNO;

NON EQUI JOIN: If we join tables with any condition other than equality condition then we call as a non Equi join.

Ex: SELECT * FROM EMP, SALGRADE WHERE (SALARY > LOWSAL) AND (SALARY < HIGHSAL)

SELF JOIN: Joining a table by itself is known as self join. When we have some relation between the columns within the same table then we use self join.

When we implement self join we should use alias names for a table and a table contains any no. of alias names.

Ex: SELECT E.EID, E.ENAME, E.SALARY, M.MID, M.ENAME MANAGERSNAME, M.SALARY FROM EMP E, EMP M WHERE E.EID=M.MID.

CROSS JOIN: Cross join is used to join more than two tables without any condition we call as a cross join. In cross join each row of the first table join with each row of the second table.

So, if the first table contain „m‟ rows and second table contain „n‟ rows then output will be „m*n‟ rows.

Ex: SELECT * FROM EMP, DEPT

Ex: SELECT * FROM EMP CROSS JOIN DEPT

NATURAL JOIN: It is used to avoid duplicate column from the tables.

EX: SELECT EID, ENAME, SALARY, DNO, DNAME FROM EMP E, DEPT D WHERE E.EID=D.DNO

*/